

- uncertainty growth over time,
 - risk curves such as collision probability over time,
 - success and failure case studies.
- xvii) Discuss why pedestrian trajectory prediction is inherently uncertain, when deterministic prediction fails, whether the probabilistic model captures meaningful ambiguity, how reliable the uncertainty estimates are, and when uncertainty-aware prediction improves risk assessment.

Bonus

Extend the project to a more realistic setting, such as explicitly modeling social interactions between nearby pedestrians, using multimodal prediction with mixture models or multiple futures, incorporating map awareness or obstacles, or performing a detailed calibration study to determine whether the uncertainty estimates are overconfident or underconfident.

Based on the above results, identify the model that gives the best trade-off between trajectory accuracy, uncertainty quality, risk estimation, and computational cost. Record all datasets, pre-processing choices, architectures, losses, sampling methods, risk metrics, horizons, and experimental settings you tried.

Imitation Learning / Control

10. Diffusion-Based Control from Demonstrations

Part A

- i) Choose a control or manipulation task where demonstration data can be generated or obtained. Possible examples include reaching, goal-directed motion, trajectory tracking, pushing, simple manipulation, 2D navigation, or any environment where trajectories of states and actions can be collected.

- ii) Define the demonstration dataset as trajectories of the form

$$(s_1, a_1, s_2, a_2, \dots, s_T, a_T).$$

Demonstrations may come from an expert policy, a scripted controller, human-designed behavior, or another reliable source.

- iii) Define the learning objective as either learning a policy

$$\pi(a_t \mid s_t)$$

or a short-horizon trajectory/action generator

$$\pi(a_{t:t+H} \mid s_t).$$

- iv) Implement a **behavior cloning (BC)** baseline. Train a neural network to predict actions from states using supervised learning, such as MSE for continuous actions or cross-entropy for discrete actions.

- v) Evaluate the behavior cloning policy in the environment. Record rollout performance, success rate, trajectory quality, and robustness to perturbations. Discuss limitations such as compounding errors and inability to represent multiple valid actions.

Part B

- vi) Implement a **diffusion model** for action generation. Define a forward process that gradually adds noise to actions or action sequences, and a reverse process that learns to denoise them.
- vii) Train the diffusion model to generate actions or short action sequences conditioned on the current state, and optionally on a goal:

$$p(a_{t:t+H} \mid s_t).$$

- viii) Implement the sampling procedure so that the model can generate actions from noise. The model may generate a single action at each timestep or a short sequence of future actions.
- ix) Use the diffusion model as a control policy. At each decision step, sample an action, or sample a short action sequence and execute the first action.
- x) Evaluate the diffusion policy in the same environment and under the same conditions as the behavior cloning baseline. Record success rate, trajectory quality, stability, smoothness, robustness to noise, and computational cost.

Part C

- xi) Compare behavior cloning and diffusion-based control across multiple rollouts and random seeds.
- xii) Analyze whether the diffusion model captures multiple valid behaviors, especially in states where several actions could lead to success.
- xiii) Study when diffusion-based control outperforms or underperforms behavior cloning. Consider task difficulty, amount of demonstration data, action dimensionality, sampling time, and robustness to perturbations.
- xiv) Produce plots and visual results showing:
 - example trajectories for BC and diffusion policies,
 - success rate comparison,
 - action distributions,
 - generated trajectories or action sequences from noise,
 - performance versus training steps,
 - computational cost or sampling time.
- xv) Discuss why diffusion models may be useful for control, whether they capture multi-modal behavior, what computational trade-offs they introduce, and what limitations appeared in your setup.

Bonus

Extend the project to a more advanced diffusion-control setting, such as goal-conditioned diffusion, comparison with Flow Matching, trajectory-level diffusion instead of single-step action generation, or a study of how the prediction horizon length affects performance.

Based on the above results, identify the method that gives the best trade-off between success rate, robustness, trajectory quality, multi-modality, and computational cost. Record all environments, demonstration sources, architectures, horizons, diffusion settings, baselines, random seeds, and experimental choices you tried.

11. Flow Matching for Imitation Learning

Part A

- i) Choose a control or sequential decision-making task where demonstration data can be collected. Possible examples include 2D navigation, reaching, tracking, simple manipulation, trajectory following, or any task with state-action trajectories.
- ii) Collect demonstration data from an expert controller, scripted policy, synthetic trajectory generator, or another reliable source. Each demonstration should contain trajectories of the form

$$(s_1, a_1, s_2, a_2, \dots, s_T, a_T).$$

- iii) Define the learning objective as generating actions or short action sequences conditioned on the current state:

$$\pi(a_{t:t+H} \mid s_t).$$

- iv) Implement a **behavior cloning (BC)** baseline. Train a neural network to predict actions from states using supervised learning, such as mean squared error for continuous actions.
- v) Evaluate the behavior cloning policy in the environment. Record success rate, trajectory quality, stability, and robustness to perturbations. Discuss common limitations such as compounding error and mode collapse.

Part B

- vi) Implement a **flow matching** model that learns a continuous-time vector field transporting samples from noise to action sequences or trajectories:

$$\frac{dz_t}{dt} = v_\theta(z_t, t, \text{context}).$$

Here $z_0 \sim \mathcal{N}(0, I)$, z_1 corresponds to the target action sequence or trajectory, and the context should include the current state, and optionally the goal.

- vii) Define a path between noise and data, for example the linear interpolation

$$z_t = (1 - t)z_0 + tz_1.$$